Karnataka ReddyJana Sangha(R)
VEMANA INSTITUTE OF TECHNOLOGY
Approved by AICTE-New Delhi, Affiliated to VTU-Belagavi, Recognized by Govt. of Karnataka
#1,Mahayogi Vemana Road, 3rd Block, Koramangala, Bengaluru - 34.
www.vemanait.edu.in
DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

Module-2: File attributes and permissions, The shells interpretive cycle, Shell programming

File attributes and permissions: The ls command with options. Changing file permissions: the relative and absolute permissions changing methods. Recursively changing file permissions. Directory permissions.

The shells interpretive cycle: Wild cards. Removing the special meanings of wild cards. Three standard files and redirection. **Connecting commands:** Pipe. Basic and Extended regular expressions. The grep, egrep. Typical examples involving different regular expressions.

Shell programming: Ordinary and environment variables. The .profile. Read and readonly commands. Command line arguments. exit and exit status of a command. Logical operators for conditional execution. The test command and its shortcut. The if, while, for and case control statements. The set and shift commands and handling positional parameters. The here (<<) document and trap command. Simple shell program examples.

VTUSYNC.IN

Text Book:

Sumitabha Das., Unix Concepts and Applications., 4thEdition., Tata McGraw Hill (Chapter 1,2 ,3,4,5,6,8,13,14)

File attributes and permissions

File Attributes:

- ls command can be used to obtain a list of all filenames in the current directory.
- -l option can be used to obtain a detailed list of attributes of all files in the current directory.

Example:

$ ls -l

Type & Perm	Link	Owner	Group	Size	Date & Time	File Name
-rwxr-xr--	1	kumara	metal	195	may 10 13:45	chap01
drwxr-xr-x	2	kumar	metal	512	may 09 12:55	helpdir

1.Type & Perm

- This represents the file-type and the permission given on the file.

File-Type

The first character indicates type of the file as shown below:

i) hyphen (-) for regular file ii) d for Directory file

iii) l for Symbolic link file iv) b for block special file v) c for character special file

File Permission

The remaining 9 characters indicates permission of the file.

There are 3 permissions: read (r), write (w), execute (x).

i) Read: Grants the capability to read, i.e., view the contents of the file.

ii) Write: Grants the capability to modify, or remove the content of the file.

iii) Execute: User with execute permissions can run a file as a program.

There are 3 types of users: owner, groups and others.

- The permission is broken into group of 3 characters:

i) The first 3 characters (2-4) represent the permissions for the file's owner.

ii) The middle 3 characters (5-7) represent the permissions for the group to which the file belongs.

iii) The last 3 characters (8-10) represents the permissions for everyone else.

VTUSYNC.IN

2) Link

• This indicates the number of file names maintained by the system.

• This does not mean that there are so many copies of the file. (Link similar to pointer).

3) Owner

• This represents the owner of the file. This is the user who created this file.

4) Group

• This represents the group of the owner.

• Each group member can access the file depending on the permission assigned.

• The privileges of the group are set by the owner of the file and not by the group members.

• When the system admin creates a user account, he has to assign these parameters to the user:

i) The user-id (UID) and ii) The group-id (GID)

5) Size

• This represents the file size in bytes.

• It is the number of character in the file rather than the actual size occupied on disk.

6) Date & Time

• This represents the last modification date and the time of the file.

• If you change only the permissions /ownership of the file, the modification time remains unchanged.

• If at least one character is added/removed from the file then this field will be updated.

7) File name

• This represents the file or the directory name

The ls command with options

VTUSYNC.IN

- This command can be used to list the files and directories stored in the current directory.

Syntax:

ls [options] [argument]

example:

$ ls

bin lib users work

With options it can provide information about the size, type of file, permissions, dates of file creation, change and access.

Ls command Options

a	Lists all files,including those beginning with a dot (.)
d	Lists only names of directories, not the files in the directory
F	Indicates type of entry with a trailing symbol: 1. Executables with * 2. Directories with / and 3. Symbolic link ith @
R	Recursive list
u	Sorts filenames by last access time
t	Sorts filenames by last modification time
i	Displays inode number
l	Long listing: lists the mode, link information, owner, size, last modification (time). If the file is a symbolic link, an arrow (->) precedes the pathname of the linked to file.

- -l option can be used to get more information about the listed files.
- The mode field is given by the -l option and consists of 10 characters.

Character	If entry is a
d	directory
-	Plain file
b	Block-type special file
c	Character-type special file
l	Symbolic link
s	socket

VTUSYNC.IN

$ ls -l

Type & Perm	Link	Owner	Group Size	Date & Time	File Name
-rwxr-xr--	1	kumara	metal 195	may 10 13:45	chap01
drwxr-xr-x	2	kumar	metal 512	may 09 12:55	helpdir

Listing Directory Attributes

• $ls -d: can be used to list only names of directories, not the files in the directory.

• However, this command will not list all subdirectories in the current directory.

Example:

$ls –ld helpdir progs

drwxr-xr-x	2	kumar	metal 512	may 9 10:31	helpdir
drwxr-xr-x	2	kumar	metal 512	may 9 09:57	progs

• Directories are easily identified in the listing by the first character(d) of the first column.

Changing file permissions

- A file is created with a default set of permission.
- This command can be used in two ways: 1) Relative mode and 2) Absolute mode.

Relative Permissions

This command can be used to add/delete permission for specific type of user (owner, group or others).

• This command can be used to

→ change only those permissions specified in the command line and

→ leave the other permissions unchanged.

Syntax:

chmod category operation permission filename

This command takes 4 arguments:

1) category can be

VTUSYNC.IN

u → user (owner)

g → group

o → others

a → all (ugo)

2) operation can be

+ → assign

- → remove

= → absolute

3) permission can be

r → read

w → write

x → execute

4) Filename whose permission has to changed

Example:

$ ls -l xstart

-rw-r—r-- 1 kumar metal 1906 sep 23:38 xstart

$ chmod u+x xstart // user (u) is added(+) an execute(x) permission

$ ls -l xstart

-rwxr—r-- 1 kumar metal 1906 sep 23:38 xstart

$ chmod ugo+x xstart OR chmod a+x xstart // all(a) are added (+) an execute(x) permission

$ ls –l xstart

-rwxr-xr-x 1 kumar metal 1906 sep 23:38 xstart

$ chmod go-r xstart // group(g) & others(o) are removed (-) a read(r) permission

$ls –l xstart

VTUSYNC.IN

-rwx—x--x 1 kumar metal 1906 sep 23:38 xstart

• This command can also accept multiple file names.

Example:

$ chmod u+x note1 note2 note3

Recursively Changing File Permissions

chmod command can be used to change recursively permission of all the files and subdirectories found in the current directory.

Example:

chmod -R a+x c_progs // current directory c_progs

Here, all files and subdirectories are made executable for all users in current directory c_progs.

Absolute Permissions

- This command can be used to add/delete permission for all type of users (owner, group or others).
- This command can be used to change all permissions specified in the command line.

Syntax:

chimed octal_value filename

This command takes 2 arguments:

- octal_value contains 3 octal digits to represent 3 type of users
- Filename whose permission has to changed.

- **octal_value contains 3 octal digits to represent 3 type of users (owner, group or others).**
  - First digit is for user
  - Second digit is for group and
  - Third digit is for others

Each digit represents a permission as shown below:

- 4 (100) – read only
- 2 (010) – write only

VTUSYNC.IN

- 1 (001) - execute only
- 6 (110) – read & write only
- For ex: octal_value of 644(110 100 100) means
    - user can read & write only
    - group can read only
    - others can read only

Example

$ls -l xstart
-rw-r--r-- 1 kumar metal 1906 sep 23:38 xstart // current permission 664

$chmod 744 xstart same as **$chmod u+x xstrat**
//user(u) is added(+) and execute(x) permisssion

$ls -l xstart
-rw-r--r-- 1 kumar metal 1906 sep 23:38 xstart // current permission 664

$chmod 755 xstart same as **$chmod a+x xstrat**
//all(a) are added(+) and execute(x) permisssion

$ls -l xstart
-rwxr-xr-x 1 kumar metal 1906 sep 23:38 xstart

$chmod 711 xstart same as **$chmod u+x xstrat**
//user(u) is added(+) and execute(x) permisssion

$ls -l xstart
-rwx--x--x 1 kumar metal 1906 sep 23:38 xstart

$chmod 711 xstart same as **$chmod go-r xstrat**
//group(g) & others(o) are removed a red(r) permission

$ls -l xstart
-rwx--x--x 1 kumar metal 1906 sep 23:38 xstart

- It is the directory permissions that determine whether a file can be deleted or not.

i) 777 signify all permissions for all categories, but still we can prevent a file from being deleted.

ii) 000 signifies absence of all permissions for all categories, but still we can delete a file.

• The system administrator can change the file permissions of every user.

i) Only owner can change the file permissions.

ii) User cannot change other user‘s file‘s permissions.

VTUSYNC.IN

Relative Permissions	Absolute Permissions

$ ls-l xstart
-rw-r—r—l kumar metal 1906 sep 23:38 xstart
$ chmod u+x xstart
$ ls-l xstart
-rwxr—r—l kumar metal 1906 sep 23:38 xstart
$ chmod ugo+x xstart OR chmod a+x xstart
$ ls -l xstart
-rwxr-xr-x l kumar metal 1906 sep 23:38 xstart
$ chmod go-r xstart
$ls -l xstart
-rwx—x—x l kumar metal 1906 sep 23:38 xstart
	$ ls-l xstart
	-rw-r—r—l kumar metal 1906 sep 23:38 xstart
	$ chmod 744 xstart
	$ ls-l xstart
	-rwxr—r—l kumar metal1906 sep 23:38 xstart
	$ chmod 755 xstart OR chmod a+x xstart
	$ ls-l xstart
	-rwxr-xr-x l kumar metal 1906 sep 23:38 xstart
	$ chmod 711 xstart
	$ ls-l xstart
	-rwx—x—l kumar metal 1906 sep 23:38 xstart

Directory Permissions

- It is possible that a file cannot be accessed even though it has read permission, and can be removed even when it is write protected.
- The default permissions of a directory are rwxr-xr-x (755)
- A directory must never be writable by group and others.

Example:

$mkdir c_progs

$ls –ld c_progs

drwxr-xr-x 2 kumar metal 512 may 9 09:57 c_progs

- If a directory has write permission for group and others also, be assured that every user can remove every file in the directory. As a rule, you must not make directories universally writable unless you have definite reasons to do so.

Directory Permissions

Command

Syntax:

chmod category operation permission directoryname

$ chmod –r pcd -> removes read permission

$ chmod +r pcd -> adds read permission

VTUSYNC.IN

$chmod +w pcd -> adds write permission

$chmod 555 pcd-> removes write permission

Changing File Ownership

- Usually, on BSD and AT&T systems, there are two commands meant to change the ownership of a file or directory.
- Let kumar be the owner and metal be the group owner.
- If sharma copies a file of kumar, then sharma will become its owner and he can manipulate the attributes.
- chown changing file owner and chgrp changing group owner
- On BSD, only system administrator can use chown
- On other systems, only the owner can change both

Chown

- Changing ownership requires super user permission, so use su command

$ls -l note

-rwxr----x 1 kumar metal 347 may 10 20:30 note

$chown sharma note; ls -l note

-rwxr----x 1 sharma metal 347 may 10 20:30 note

- Once ownership of the file has been given away to sharma, the user file permissions that previously applied to Kumar now apply to sharma. Thus, Kumar can no longer edit note since there is no write privilege for group and others. He cannot get back the ownership either. But he can copy the file to his own directory, in which case he becomes the owner of the copy.

Chgrp

- This command changes the file‘s group owner. No super user permission is required.

#ls –l dept.lst

VTUSYNC.IN

-rw-r—r-- 1 kumar metal 139 jun 8 16:43 dept.lst

#chgrp dba dept.lst;

ls –l dept.lst -rw-r—r-- 1 kumar dba 139 Jun 8 16:43 dept.lst

Soft Link and Hard Link

What is Soft Link And Hard Link

- A symbolic or soft link is an actual link to the original file.
- If you delete the original file, the soft link has no value, because it points to a non-existent file.
- Hard link is a mirror copy of the original file
- Even if you delete the original file, the hard link will still has the data of the original file.
- Because hard link acts as a mirror copy of the original file.

Command for creating Soft link

$ cat > source.file

$ cat source.file

Welcome to OSTechNix

$ ln –s source.file softlink.file

$ cat source.file

Welcome to OSTechNix

$ cat softlink.file

Welcome to OSTechNix

$ ls –lia

Command for Hard link

$ cat > source.file

$ cat source.file

VTUSYNC.IN

Welcome to OSTechNix

$ ln source.file hardlink.file

Difference between Hardlink and soft link:

PARAMETERS	HARD LINK	SOFT LINK
Inode number	Files that are hard linked take the same inode number.	Files that are soft linked take a different inode number
Directories	Hard links are not allowed for directories linking directories (Only a superuser can do it)	Soft links can be used for linking directories
File system	It cannot be used across file systems.	It can be used across file systems.
Data	Data present in the original file will still be available in the hard links	Soft links only point to the file name, it does not retain of the file data
Original file's deletion	If the original file is removed, the link will still work as it accesses the data of the original was having access to	If the original file is removed, link will not work as it doesn't accesses the original file

VTUSYNC.IN

THE SHELLS INTERPRETIVE CYCLE, CONNECTING COMMANDS

The shells interpretive cycle

The shell performs following activities in its interpretive-cycle:

1) The shell

→ issues the prompt ($) and

→ waits for user to enter a command (like ls chap*).

2) After a command is entered, the shell

→ scans the command-line for metacharacters (like 'ls chap*') and

→ expands the abbreviations to recreate a simplified command-line ('ls chap1 chap2').

3) Then, the shell

→ passes the command-line to the kernel for execution and

→ waits for the command to complete its task.

4) After the command is executed, the shell

→ issues the prompt ($) again and

→ waits for the user to enter a next command.

VTUSYNC.IN

Wild Cards

- The metacharacters that are used to construct the generalized pattern for matching filenames belong to a category called wild-cards.

Wild Card/ Character class	Match
*	Any number of characters including none
?	A single character
[ijk]	A single character either an i, j or k
[x-z]	A single character that is within the ASCII range of the characters x and z
[!ijk]	A single character that is not an i, j, or k
[!x – z]	A single character that is not within the ASCII range of the characters x and z

Metacharacter *

The metacharacter "*" matches any number of characters including none.

Examples:

$ ls chap* // To list all files that begin with "chap" chap chap01 chap02

chap03 chap04 chapx chapy chapz

- When the shell encounters this command line, it identifies the * immediately as a wild-card. It then looks in the current directory and recreates the command line as below

$ ls chap chap01 chap02 chap03 chap04 chapx chapy chapz

- The shell now hands over this command to the kernel which uses its process creation facilities to run the command.
- The metacharacter ? matches a single character.

Metacharacter ?

- The metacharacter ? matches a single character.

Example:

$ ls chap? // To list all five-characters filenames beginning with "chap"

chapx chapy chapz

$ ls chap?? // To list six-characters filenames beginning with "chap" chap01

chap02 chap03

- Both * and ? operate with some restrictions.
- The * doesn't match all filenames beginning with a dot (.) or forward slash(/).

Example:

$ ls .C* // to list all C extension filenames

$ cd /usr?local // this doesn't match /usr/local

Character Class

The character class comprises a set of characters enclosed by the rectangular brackets, [and],.

VTUSYNC.IN

• The character class matches a single character in the class.

Example:

Character Class	Match
[ijk]	A single character either an i, j or k
[x-z]	A single character that is within the ASCII range of the characters x and z.
chap0[124]	chap01, chap02, chap04
[!ijk]	A single character that is not an i, j, or k
[!x – z]	A single character that is not within the ASCII range of the characters x and z

This can be combined with any string or another wild-card expression.

Example:

$ ls chap0[124] //Matches chap01, chap02, chap04 and lists if found.

$ ls chap[x-z] //Matches chapx, chapy, chapz and lists if found.

Negating the Character Class (!)

- Not operator (!) can be used to negate the character class.

Example

$ls [!a-zA-Z]* //To match all filenames that don't begin with an alphabetic character.

Matching Totally Dissimilar Patterns

$ cp $HOME/prog_sources/*.{c,java} . // To copy all the C and Java Souce programs from another directory to the current directory

$ cp /home/srm/{project,html,scripts/* . // To copy all files from 3 directories (project, html and scripts) to the current directory.

Removing the Special Meanings of Wild Cards (Escaping and Quoting)

Escaping is providing a \ (backslash) before the wildcard to remove its special meaning.

Example:

$ rm chap* // to remove file named "chap*" "\" is used to suppress special meaning of *

$ cat chap0\[1-3\] // to list the contents of the file named "chap0[1-3]"

$ rm My\ Document.doc // to remove file named "My Documend.doc"

$ echo \\ // outputs \

• Quoting is enclosing the wild-card within quotes to remove its special meaning.

• When a command argument is enclosed in quotes, the meanings of all enclosed special characters are turned off.

Example:

$ rm 'chap*' // to remove file named "chap*"

$ rm "My Document.doc" // to remove file named "My Documend.doc"

$ echo "\" // outputs "\"

VTUSYNC.IN

Redirection : Three Standard Files

- The shell associates three standard files with the terminal:

  → two for display and

  → one for the keyboard.

- When a user logs in, the shell makes available three standard files.
- Each standard file is associated with a default device:

1) Standard input: The file representing input which is connected to the keyboard.

2) Standard output: The file representing output which is connected to the display.

3) Standard error: The file representing error messages that come from the command or shell

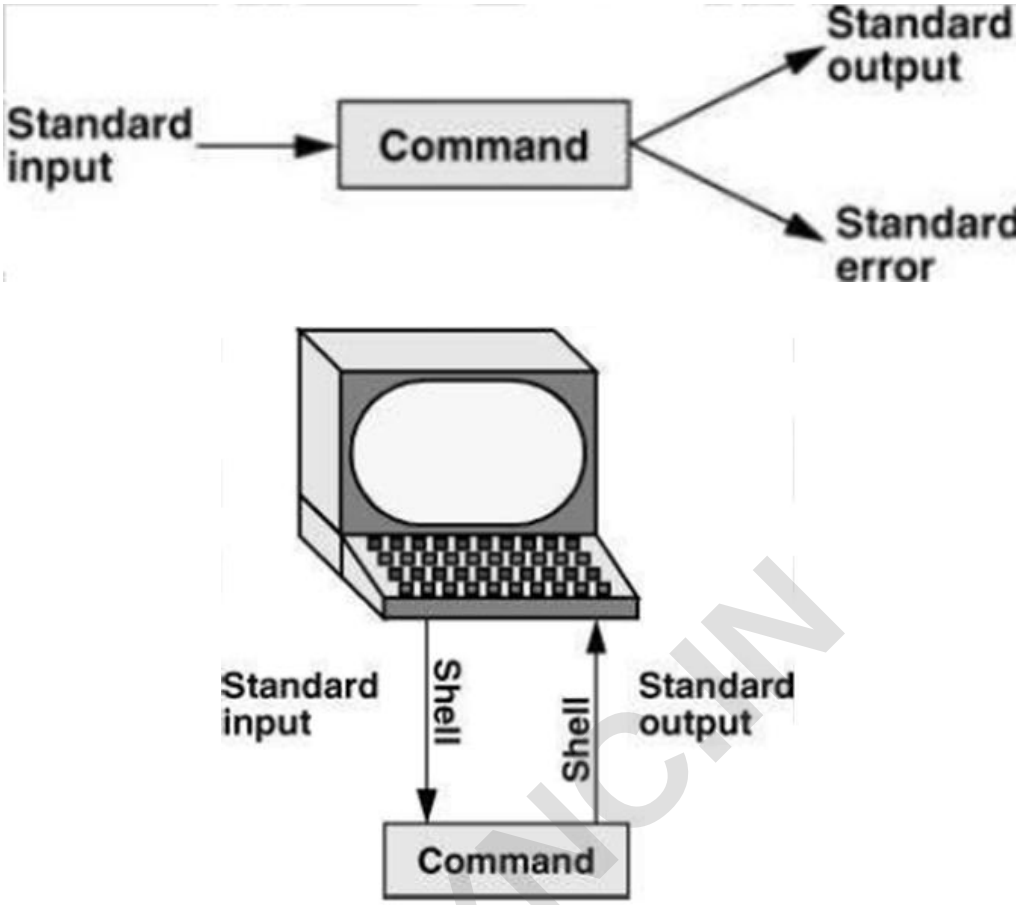


Fig. Standard input, output and error

Standard Input

The standard input can represent three input sources:

1) The keyboard, the default source.

2) A file using redirection with the < symbol.

3) Another program using a pipeline.

• By default, the shell directs standard input from the keyboard.

Example:

$wc

hello

world

<ctrl+d> // end of input

2 2 10 // output

VTUSYNCIN

- The redirect input symbol (<) instructs the shell to redirect a command's input to come from the specified file instead of from the keyboard.

Example: $ cat sample.txt

hello

world

$ wc < sample.txt

2 2 10 // output

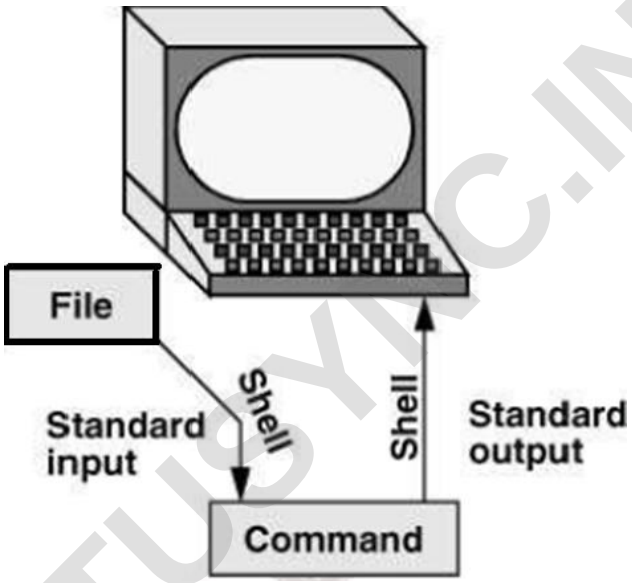


Fig. Standard input.

VTUSYNC.IN

Standard Output

• The standard output can represent three possible destinations:

1) The terminal, the default destination.

2) A file using the redirection symbols > and >>.

3) As input to another program using a pipeline.

• By default, the shell directs standard output from a command to the screen.

Example:

$ cat sample.txt

hello

world

$ wc sample.txt

2 2 10 //Output

• The redirect output symbol (>) instructs the shell to redirect the output of a command to the specified file instead of to the screen.

Example:

$ wc sample.txt > temp.txt

$ cat temp.txt

2 2 10 // output of wc stored in temp.txt

• >> can be used to append to a existing file.

VTUSYNC.IN

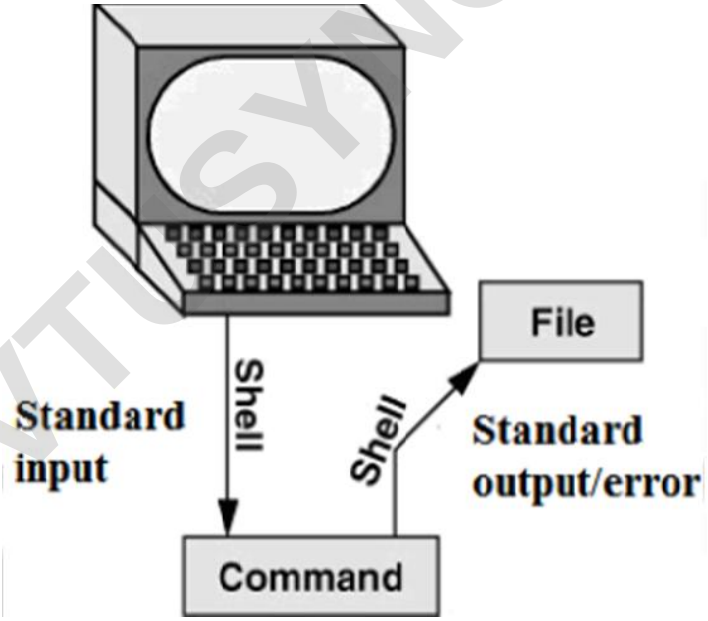


Fig. Standard output and error.

Standard Error

- By default, the shell directs standard error from a command to the screen.

Example:

$ cat empty.txt

cat: cannot open empty.txt // error because empty.txt is non existent file

• The redirect output symbol (>) instructs the shell to redirect the error messages of a command to the specified file instead of to the screen.

Example:

$ cat empty.txt >> errorfile.txt

$ cat errorfile.txt

cat: cannot open empty.txt // error message of cat stored in errorfile.txt

Pipe : Connecting Commands

- Pipe is another form of output redirection.
- With piping, the output of a command can be used as input (piped) to a subsequent command.

Syntax:

$ command1 | command2

- Here, Output from command1 is piped into input for command2.
- The symbol '|' denotes a pipe.

Example:

$ ls -l | lp

- Here pipe is used to send the output of ls to the lp to print a hard copy of the listing of the current directory.
- Pipeline can be used with 2 or more commands. But the user should know the behavioral properties of each commands to place them there.

Example:

$ls -l | wc -l | lp // to print a count of files in current directory

When a Command Needs to be Ignorant of its Source

If we wish to find total size of all C programs contained in the working directory, we can use the command:

$ wc –c *.c

VTUSYNC.IN

- However, it also shows the usage for each file(size of each file). We are not interested in individual statistics.
- Solution: We must make wc ignorant of its input source. So, feed the concatenated output stream of all the .c files to wc –c as its input:

$ cat *.c | wc –c

tee : Splitting the Output

- This is an external command that handles a character stream by duplicating its input.

• This command

→ saves one copy in a file and

→ writes the other copy to standard output.

• This command is also a filter and hence can be placed anywhere in a pipeline.

Example:

$ who | tee users.lst //to display the output of who and save this output in a file users.lst

Command Substitution

- Command substitution allows to substitute the output of one command into a given command line.

Syntax:

`command`

• Here, the command must be enclosed between backquotes.

Example:

$ echo Current date and time is `date`

Current date and time is Sat Nov 24 10:12 IST 2017

$ echo "There are `ls | wc –l` files in the current directory"

VTUSYNC.IN

There are 3 files in the current directory

grep

- g/re/p means “globally search for a regular expression and print all lines containing it”.

• This command can be used to search a file(s) for lines that have a certain pattern.

• This command

→ scans the file for a pattern and

→ displays

i) lines containing the pattern or

ii) line numbers

Syntax:

$grep pattern file(s)

Example:

$grep “MH” student.lst // display lines containing "MH" from the file student.lst

• Patterns with and without quotes is possible.

• Quote is mandatory when pattern involves more than one word.

Example:

$grep “My Document” student.lst // display lines containing "My Document" from student.lst

• This command can be used with multiple filenames.

Example:

$grep “MH” student.lst vtu.lst rank.lst

grep Options

VTUSYNC.IN

Unix supports below listed options:

Option	Significance
-i	Ignores case for matching
-v	Doesn't display lines matching expression
-n	Displays line numbers along with lines
-c	Displays count of number of occurrences
-l	Displays list of filenames only
-e	Exp specifies expression with this option
-x	Matches pattern with entire line
-f	Takes patterns from file, one per line
-E	Treats pattern as an extended regular expression (ERE)
-F	Matches multiple fixed strings (in **fgrep**-style)

Ignores Case (-i)

- -i (ignore) option can be used to search all lines containing a pattern regardless of uppercase and lowercase distinction

Example:

$ grep -i "MH" student.lst // matches all the words such as MH mH Mh mh

4 | MH | 10 | IS | 111

4 | MH | 11 | CS | 401

Deleting Lines (-v)

- -v option can be used to print all lines that do not contain the specified pattern in a file.

• Example:

$grep -v 'MH' student.lst

4 | GW | 11 | CS | 402

VTUSYNC.IN

4 | VV | 11 | CS | 403

Displaying Line Number (-n)

• -n (line number) option can be used to display line numbers containing the pattern.

Example:

$grep -n 'MH' student.lst

1 4 | MH | 10 | IS | 111

2 4 | MH | 11 | CS | 401

Counting Lines Containing the Pattern(-c)

• -c (line count) option can be used to count number of lines containing the pattern.

Example:

$grep -c 'MH' student.lst

2

Displaying Filenames(-l)

• -l (list filename) option can be used to list out the files containing the pattern.

Example:

$grep -l 'MH' *.lst

Matching Multiple Pattern(-e)

• -e option can be used to match multiple pattern in a file.

Example:

$grep -e 'MH' -e 'VV' student.lst

Taking Pattern From File(-f)

• -f option can be used to place all matched pattern in a separate file, one pattern per line.

$cst>pat

MH

VTUSYNC.IN

VV

WW

$grep -f pat student.lst

Basic Regular Expression (BRE)

- grep uses an expression of a different type to match a group of similar patterns.

• This command

→ uses an elaborate metacharacter set and

→ can perform amazing matches.

• If an expression uses metacharacters, it is termed a regular expression.

• Regular expression can be classified as

1) BRE (Basic Regular Expression) and

2) ERE (Extended Regular Expression)

• grep supports

→ BRE by default and

→ ERE with the –E option.

Character subset of BRE is listed below:

Metacharacter/ Character class	**Match**
*	Zero or more occurrences of the previous character
g*	Nothing or ggg,ggg,etc
.	A single character
.*	Nothing or any number of characters
[pqr]	A single character p,q or r
[c1-c2]	A single character within the ASCII range represented by c1 and c2
^[pqr]	a single character which is not p,q or r

VTUSYNC.IN

^Pattern	Pattern at beginning of line
Pattern$	pattern at end of line
^$	Line containing nothing

Character Class

• The character class comprises a set of characters enclosed by the rectangular brackets, [and].

• The character class matches a single character in the class.

• When you use range, make sure that the character on the left of the hyphen has a lower ASCII value than the one on the right.

• caret (^) can be used to negate the character class.

Example:

Character Class	**Match**
[pqr]	a single character p,q or r
[x-z]	A single character between characters x to z.
^[pqr]	a single character whuch is not p, q or r

Example:

$ grep -I "[Mm][Hh]" demo_file // matches all the words such as
MH mH Mh mh

Asterisk (*)

• The asterisk (*) refers to the immediately preceding character.

• It indicates zero or more occurrences of the previous character. g* → nothing or g, gg, ggg, etc.

lg* → l or lg, lgg, lggg, etc.

Example:

$ grep "isaa*c" demo_file // matches isac isaac or isaaac

Dot (.)

VTUSYNC.IN

• A dot (.) matches a single character.

Example:

$ grep "2..." demo_file // matches all 4 character words beginning with 2

• Regular expression "**.***" → signifies any number of characters or none

Example:

$ grep "Rachitha .* " demo_file // matches pattern Rachitha,

Rachitha M V, Rachitha Venkatesh,Rachitha Hithendra

$ grep -l "Rachitha" *.* // matches all extension filenames

Specifying Pattern Locations (^ and $)

- Following two metacharcters can match a pattern at the beginning or end of a line.

Metacharcter	**Match**
^Pattern	Pattern at beginning of line
Pattern$	Pattern at end of line

- Anchoring a pattern is often necessary when it can occur in more than one place in a line, and we are interested in its occurrence only at a particular location.

Example:

- grep "^2" emp.lst //Selects lines starting with 2
- grep "7…$" emp.lst //Selects lines where salary between number b/w 7000 to 7999

Extended Regular Expression (ERE) and egrep

• ERE can be used to match dissimilar patterns with a single expression.

• This uses some additional characters as listed below:

VTUSYNC.IN

Metacharacter/ Character class	Match
ch+	one or more occurances of character ch
ch?	zero or one occurances of character ch
exp1|exp2	exp1 or exp2
(x1|x2)x3	x1x3 or x2x3

• If current version of grep doesn't support ERE, then use egrep but without the –E option.

• -E option treats pattern as an ERE.

Example:

$ grep -E "isaa*c" demo_file // matches isac isaac or issaac

$ grep –E 'vijaykumar | jayakumar' demo_file // matches multiple patterns

$ grep –E '(vijay | jaya) kumar' demo_file

VTUSYNC.IN

SHELL PROGRAMMING

• A shell script contains a list of commands which have to be executed regularly.

• Shell script is also known as shell program.

• The user can execute the shell script itself to execute commands in it.

• A shell script runs in interpretive mode. i.e. the entire script is compiled internally in memory and then executed.

• Hence, shell scripts run slower than the high-level language programs.

• ".sh" is used as an extension for shell scripts.

Example: A shell script (program1.sh) to execute few commands.

```
#! /bin/sh
echo “Welcome to Shell Programming” # print message
echo “Today’s date : `date`” # print date
echo “My Shell :$SHELL” # print shell name
```

• The hash symbol # indicates the comments in the script.

• The shell ignores all the characters that follow the # symbol. However, this does not apply to the first line.

• The first line

"#! /bin/sh" indicates the path where the shell script is available.

• There are 2 ways to execute a shell script:

1) **Execute Shell Script Using File Name**

• By default, script is not executable. So, the chmod command can be used to make the script executable.

• The scripts are executed in a separate child shell process.

VTUSYNC.IN

• The child shell reads and executes each statement in interpretive mode.

Run:

$ chmod +x program1.sh // add executable permission

$ program1.sh // execute the script program1.sh need to set PATH variable to current

directory **$export PATH=$PATH:. OR $PATH=$PATH:.**

Output:

Welcome to Shell Programming

Today's date: Mon Nov 4 11:02 :45 IST 2017

My Shell: /bin/sh

2) Execute Shell Script by Specifying the Interpreter

The user can also execute a shell script by specifying the interpreter in the command line.

- Here, the script neither requires a executable permission nor an interpreter line.

Run:

$ sh program1.sh //Execute using sh interpreter

$ bash program1.sh //Execute using bash interpreter

Output:

Welcome to Shell Programming

Today's date: Mon Nov 4 11:02:45 IST 2017

My Shell: /bin/sh

VTUSYNC.IN

Ordinary and Environment Variables

• Shell variables are of 2 types: 1) Environment and 2) Ordinary

Environment Variable

• Environmental variables are used to provide information to the programs you use.

• These variables control the behavior of the system.

• They determine the environment in which the user works.

• If environment variables are not set properly, the users may not be able to use some commands.

• Environment variables are so called because they are available in the user's total environment

i.e. the sub-shells that run shell scripts and mail commands and editors.

• Some variables are set by the system, others by the users, others by the shell programs.

• env command can be used to display environment variables.

Example:

$ env

HOME=home/kumar IFS=' '

LOGNAME=kumar

MAIL= /var/mail/kumar

MAILCHECK=60

PATH=/bin:/usr/bin

PS1='$'

PS2='>'

SHELL=/usr/bin/bash

TERM= tty1

1) HOME

• This variable indicates the home directory of the current user.

VTUSYNC.IN

• This variable is set for a user by the system admin in /etc/passwd.

2) IFS

• This variable contains a string of characters that are used as word separator in the command line.

• The string normally consists of the space, tab and newline characters.

3) LOGNAME

• This variable shows the username.

4) MAIL

• This variable specifies the path to user's mailbox.

5) MAILCHECK

• This variable determines how often the shell checks the file for the arrival of new mail.

6) PATH

• This variable specifies the locations in which the shell should look for commands.

• Usually, the PATH variable can be set as follows:

$PATH=/bin:/usr/bin

7) PS1 and PS2

• The shell has 2 prompts:

1) The primary prompt $ is the one the user normally sees on the monitor. $ is stored in PS1.

- The user can change the primary prompt as follows:

  $ PS1="C>"

  C> //similar to windows

2) The secondary prompt > is stored in PS2.

8) SHELL

• This variable specifies the current shell being used by the users.

• Different types of shells are:

VTUSYNC.IN

1) Bourne shell /bin/sh 2) C-shell /bin/csh 3) Korn shell /bin/ksh

• This variable is set for a user by the system admin in /etc/passwd.

9) TERM

• This variable indicates the terminal type that is used.

• Every terminal has certain characteristics that are defined in a separate control file in the terminfo directory.

• If TERM is not set correctly, vi will not work and the display will be faulty.

Ordinary(or Local) Variable

• A variable is a character string to which the user assigns a value.

• The value assigned can be a number, text, filename, device, or any other type of data.

Syntax:

variable = value // variable definition

• The value of variables are stored in the ASCII format.

Example:

$ x=50

$ echo $x //displays 50

• In command line, all words that are preceded by a $ are identified and evaluated as variables.

• A variable can be removed with unset and protected from reassignment by readonly. Both are shell internal commands.

$ set count=5

$ readonly size = 10

The variables exist only for a short time during the execution of a shell script.

• The variables are local to the user's shell environment.

VTUSYNC.IN

• The variables are not available for the other scripts or processes.

• As the variables are defined and used by specific users, they are also called user-defined variables.

Uses of Local variables

1) Setting pathnames: If a pathname is used several times in a script, we can assign it to a variable and use it as an argument to any command.

2) Using command substitution: We can assign the result of execution of a command to a variable. The command to be executed must be enclosed in backquotes.

3) Concatenating variables and strings: Two variables can be concatenated to form a new variable.

Example:

$ base=foo ; ext=.c

$ file=$base$ext

$ echo $file // prints foo.c

File .profile

• A profile file is a start-up file of an UNIX user.

• This file gets executed as soon as the user logs in.

• This file is a shell script that will be present in the home directory of each user.

• The system admin provides each user with a profile with a minimum working environment.

However, the user can customize the profile as per their requirement.

i.e. The user can

→ assign suitable values to the environment variables.

→ add and modify statements in the profile file.

• This file can be any one of the two:

1) A specific file for each individual user with responsibility for the user environment.

2) A universal file for all users with responsibility for the general environment.

VTUSYNC.IN

• The user can view his ".profile" as follows:

$ cat .profile

MAIL= /var/mail/kumar

PATH=/bin:/usr/bin

PS1='$'

PS2='>'

SHELL=/usr/bin/bash

TERM= tty1

read and readonly Commands

read Command

• read command can be used for taking input from the keyboard.

• It is shell's internal tool for making scripts interactive.

Syntax:

```
read   var_name
```

• It is used with one or more variables.

• The variables are used to hold inputs given with the standard input.

Example:

A shell script (program4.sh) to read a search string and filename from the terminal.

```
#!/bin/bash
echo "What is your name?"
read   PERSON
echo "Hello, $PERSON"
```

VTUSYNC.IN

Run:

$ sh program4.sh

Output:

What is your name? RAMA

Hello, RAMA

Readonly Command

• readonly command can be used to make variables readonly i.e. the user cannot change the value of variables.

• During shell scripting, we may need a few variables, which cannot be modified.

• This may be needed for security reasons.

Syntax:

variable=value

Example:

$ readonly PI=3.14

$ echo $PI //displays 3.14

$ PI=6.12 // this will result in error

Command line arguments

- Shell scripts can accept arguments from the command line.

• ,'. Shell scripts can be run non-interactively and be used with redirection and pipelines.

• The arguments are assigned to special shell variables called shell parameters.

• The shell parameters are reserved for specific functions.

VTUSYNC.IN

• Different shell parameters:

1) $#: Stores the number of command-line arguments.

2) $0, $1, $2, $3: These are called positional parameters which represent command line arguments.

$0: Stores the filename of the current script.

$1: Stores the first argument.

$2: Stores the second argument

$3: Stores the third argument

3) $*: Stores all the arguments entered on the command line ($1 $2 ...).

4) "$@": Stores all arguments entered on the command line, individually quoted ("$1" "$2")

5) $?: Stores the exit status of the last command that was executed.

6) $$: Stores Pid of the current shell.

7) $!: Stores PID of the last background job.

Example: A shell script (program2.sh) to read and display various shell parameters from the command line.

```
#!/bin/sh

echo "Total Number of Parameters : $#"

echo "File Name: $0"

echo "First Parameter : $1"

echo "Second Parameter : $2"

echo "Quoted Values: $*"

echo "Quoted Values: $@"

$echo "Exit value: $?"
```

VTUSYNC.IN

echo "PID of current shell: $$"

Run:

$ program2.sh "RAJA RAM" "MOHAN ROY"

Output:

Total Number of Parameters : 2 File Name : program2.sh

First Parameter : RAJA RAM Second Parameter : MOHAN ROY

Quoted Values: RAJA RAM MOHAN ROY // stored as "RAJA RAM MOHAN ROY"

Quoted Values: RAJA RAM MOHAN ROY // stored as array= {"RAJA RAM" , "MOHAN ROY"}

Exit value: 0 // zero implies success

PID of current shell: 12345

exit and Exit Status of a Command

exit command can be used to terminate a program(or script).

• This command returns value which will be available to the script's parent process.

• The $? variable contains exit status of the last command executed.

• Exit status is a numerical value returned by every command upon its completion.

• A command returns an exit status of

1) zero (0) upon successful execution and

2) non-zero upon unsuccessful execution i.e. an error condition.

• Exit status can be used to devise program-logic that branches into different paths depending on success or failure of a command.

Example: A shell script to find relationship between 2 numbers.

#! /bin/usr

VTUSYNC.IN

```
x=5; y=7

test $x –eq $y; echo "5=7: $? \n"

test $x –ne $y; echo "5!=7: $? \n"

test $x –gt $y; echo "5>7: $? \n "

test $x –ge $y; echo "5>=7: $? \n "

test $x –lt $y; echo "5<7: $? \n "

test $x –le $y; echo "5<=7: $? \n"
```

Output:

```
5=7: 1  // Returns nonzero exit status i.e. failure → False

5!=7: 0// Returns zero exit status i.e. success → True

5>7: 1  //False

5>=7: 1          //False

5<7: 0  // True

5<=7: 0  // True
```

VTUSYNC.IN

Logical Operators for Conditional Execution

• Two logical operators can be used for conditional execution: 1) && and 2) ||

1) && Operator

Syntax:

```
cmd1 && cmd2
```

• Here, cmd2 gets executed only when cmd1 succeeds.

2) || Operator

Syntax:

cmd1 || cmd2

• Here, cmd2 gets executed only when cmd1 fails.

Example: A script to illustrate the usage of && and ||.

$ cat student.lst

4 | MH | 10 | IS | 111

4 | MH | 11 | CS | 401

4 | GW | 11 | CS | 402

4 | VV | 11 | CS | 403

$ grep 'VV' student.lst && echo "Pattern found"

4 | VV | 11 | CS | 403

Pattern found

$ grep 'ZZ' student.lst || echo "Pattern not found"

Pattern not found

test Command and its Shortcut

• if-construct cannot directly handle the true or false value returned by evaluation of an expression.

• So, test command can be used to handle the true or false value returned by evaluation of an expression.

• Test command

→ uses certain operators to evaluate the condition on its right and

→ returns either a true or false exit status.

• Then, if-construct uses the exit status for making decisions.

• Test command

→ does not display any output

→ sets the parameter $? (exit status).

• Test command works in 3 ways:

VTUSYNC.IN

1) Compare two numbers.

2) Compares two strings or a single one for a null value.

3) Checks files attributes.

Numeric Comparison

Operator	Meaning
-eq	Equal to
-ne	Not equal to
-gt	Greater than
-ge	Greater than or equal to
-lt	Less than
-le	Less than or equal

Syntax:

test $op1 -operator $op2

• Operators always begin with a – (Hyphen) followed by a two-character word.

• Numeric comparison can be done on integer values only. (The decimal values are truncated).

Shorthand for test

• [and] can be used instead of test.

test $x –eq $y is equivalent to [$x –eq $y]

Example: A shell script to find relationship between 2 numbers.

```
#! /bin/usr

x=5; y=7

test $x –eq $y; echo "5=7: $? \n"

test $x –ne $y; echo "5!=7: $? \n"
```

VTUSYNC.IN

test $x –gt $y; echo "5>7: $? \n "

test $x –ge $y; echo "5>=7: $? \n "

test $x –lt $y; echo "5<7: $? \n"

test $x –le $y; echo "5<=7: $? "

OUTPUT:

5=7: 1 //False

5!=7: 0 //True

5>7: 1 //False

5>=7: 1 //False

5<7: 0 //True

5<=7: 0 //True

String Comparison

• Test command is also used for testing strings.

Operator	**True if**
s1=s2	String s1=s2
s1!=s2	String s1 is not equal to s2
-n stg	String stg is not a null string
-z stg	String stg is a null string
stg	String stg is assigned and not null

Example: A shell script to check if 2 strings are equal or not.

#!/bin/sh

echo “Enter the first string: \c”

read str1

VTUSYNC.IN

```
if  [ -z "$str1" ]
then
        echo "You have not entered the string"; exit 1
echo "Enter the second string: \c"
read str2
if  [ -z "$str2" ]
then
        echo "You have not entered the string"; exit 1
if  [ $str1 = $str2 ]
then
        echo "Both strings are equal"
else
        echo "Strings are unequal"
fi
fi
fi
```

Output:

Enter the first string: MAM

Enter the second string: MAM Both strings are equal

File Tests :

• Test command can be used to check various file attributes such as file type (-, d or l) & file permission (r, w, x).

VTUSYNC.IN

Test	True if
-e file	File exists
-f file	File exists and is a regular file
-d file	File exists and is a directory
-L file	File exists and is a symbolic link
-r file	File exists and readable
-w file	File exists and is writable
-x file	File exists and is executable
-s file	File exists and has a size greater than zero
f1 -nt f2	File f1 is newer than f2
f1 -ot f2	File f1 is older than f2
f1 -ef f2	File f1 is linked to f2

Example: A shell script (program8.sh) to check whether a file has permission for read, write and execute.

```
#! /bin/usr

echo -n "Enter file name:" read file

if [ –e $file ] ; then

        echo “File exists \n”

else

        echo “File does not exist \n”

fi

if [ -r "$file" ]

then

        echo "File is readable \n "

else
```

VTUSYNC.IN

```
        echo "File is not readable \n "
fi
if [ -w "$file" ]
then
         echo "File is writable \n "
else
        echo "File is not writable \n "
fi
if [ -x "$file" ]
then
        echo "File is executable \n "
else
        echo "File is not executable \n "
fi
```

VTUSYNC.IN

OUTPUT:

```
$ ls –l student.lst
-rw-rw-rw-   1   kumar   group   870   jun   8   15:52   student.lst
$ program8.sh Output:
Enter file name: student.lst File exists
File is readable File is writable
File is not executable
```

if Statement

• if statement is basically a “two-way” decision statement.

• This is used when we must choose between two alternatives.

• Three forms of if…else statement:

1) if...fi statement

2) if...else...fi statement

3) if...elif...else...fi statement

Syntax 1:

```
if command is successful then
execute statements
fi
```

Syntax 2:

```
if command is successful
then
        execute statements
else
        execute statements
fi
```

Syntax 3:

```
if command is successful
then
        execute statements
```

VTUSYNC.IN

```
elif command is successful
then
        execute statements
else
        execute statements
fi
```

Here is how it works:

1) If the command succeeds, the statements within then-block are executed.

2) If the command fails, the statements within else-block are executed.

Example: A script to check whether an integer is positive or negative.

```
 #! /bin/sh
echo “Enter any non zero integer: \n”
read num
if [  $num -gt 0 ]; then
        echo “Number is positive number”
else
        echo “Number is negative number”
fi
```

Output:

```
Enter any non-zero integer: 5
Number is positive number
```

VTUSYNC.IN

Program: for if else

```
#!/bin/sh
#simple if to check positive or negative number
echo "enter the number"
read num
if [  $num -gt 0  ]
then
echo "The number is positive $num "
else
echo "The number is negative $num "
fi
```

Program : To check greatest of two numbers

```
#!/bin/sh
#greatest of two number
echo "Enter the value of a and b "
read a b
if [  $a -gt $b  ]
then
        echo "a is greater"
else
        echo "b is greater"
fi
```

Program : To check greatest of three numbers using elif ladder

```
#!/bin/sh
```

VTUSYNC.IN

```
#greatest of three number

echo " Enter the value of a "

read a

echo " Enter the value of b "

read b

echo "Enter the value of c "

read c

if [  $a -gt $b  ] && [  $a -gt $c  ]

then

        echo " a is greater"

elif [  $b -gt $c  ]

then

        echo "b is greater"

else

        echo "c is greater"

fi
```

VTUSYNC.IN

case Statement

- case statement is basically a “multi-way” decision statement.

• This is used when we must choose among many alternatives.

• This also handles string tests, but in a more efficient manner than if statement.

Syntax:

```
case expression in

        pattern1) statement1 ;;
```

pattern2) statement2 ;;

pattern3) statement3 ;;

…

esac

Here is how it works:

1) Firstly the expression is matched with pattern1.

2) If the match succeeds, then statement1 will be executed.

3) If the match fails, then the expression is matched with pattern2 and this process continues.

Here is how it works:

1) Firstly the expression is matched with pattern1.

2) If the match succeeds, then statement1 will be executed.

3) If the match fails, then the expression is matched with pattern2 and this process continues.

• Each statement is terminated with a pair of semicolon (;;).

• This can match only strings but cannot handle numeric and file tests. However, this can also handle numbers but treating them as strings.

• This is very effective when the string is fetched by command substitution.

Example: A script to display appropriate message based on grades (A to D).

```
#! /bin/sh
echo “enter grade A to D: \n”
read grade
case “$grade” in
        A) echo “Excellent!” ;;
```

VTUSYNC.IN

```
        B) echo “Well done” ;;
        C) echo “You passed” ;;
        D) echo “Better try again” ;;
        *) echo “Invalid grade” ;;
esac
echo “Your grade is $grade
```

Output:

```
enter grade A to D:
 B
Well done
Your grade is B
```

Program : To perform simple calculator

```
#!/bin/sh
#simple calculator
echo " enter two numbers "
read a b
echo " Enter the operator"
echo " 1. Addition "
echo " 2. Subtraction "
echo " 3. Multiplication"
read choice
case $choice in
        1)res=`expr $a + $b` ;;
```

VTUSYNC.IN

```
        2)res=`expr $a - $b` ;;

        3)res=`expr $a \* $b` ;;

esac

echo "result : $res"
```

Matching Multiple Patterns

• case statement can also specify the same action for more than one pattern.

Example: A script to test a user response for both y and Y (or n and N).

```
 #! /bin/sh

echo “Do you wish to continue? [y/n]:”

read ans

case “$ans” in

        Y | y ) ;;

        N | n ) exit ;;

esac
```

Wild-Cards

• case statement has a superb string matching feature that uses wild-cards.

• case statement uses

→ filename matching meta-characters * and ?

→ string matching character.

Example: A script to test a user response for YES, yes, Yes, yEs (or no, NO, No, nO). #! /bin/sh

```
echo “Do you wish to continue? [y/n]:”

read ans

case “$ans” in
```

VTUSYNC.IN

```
        [Yy] [eE]* ) ;;  # Matches YES, yes, Yes, yEs, etc
        [Nn] [oO] ) exit ;;        # Matches no, NO, No, nO
        * ) echo "Invalid Response"
esac
```

expr: Evaluate an Expression

• expr command can be used to

→ evaluate an expression and

→ output the corresponding value.

• This command combines the following two functions:

1) Performs arithmetic operations on integers and

2) Manipulates strings.

1) Numeric Computation

• Five operators used on integers: +, -, *, / and %.

Syntax:

```
expr  $op1  operator  $op2
```

Example:

```
$ x=5 y=3
$ expr $x + $y // outputs 8
$ expr $x - $y // outputs 2
$ expr $x \* $y // * must be escaped to prevent shell from interpreting * as  wildcard //outputs 15
$ expr $x / $y //outputs 1
$ expr $x % $y // outputs 2
$ z =`expr $x + $y` // command substitution to assign a variable
```

VTUSYNC.IN

$ echo $z // outputs 8

2) String Handling

• Three functions used on strings:

i) Finding length of string

ii) Extracting substring

iii) Locating position of a character in a string

Syntax:

$expr "exp1" : "exp2"

• On the left of the colon (:), the string to be worked upon is placed. On the right of the colon(:), a regular expression is placed.

i) Length of the String

• The regular expression ".*" is used to print the number of characters matching the pattern.

Syntax:

$expr "string" : ".*"

Example:

$ expr “vemanait” : ‘.*’ // outputs 8

ii) Extracting a Substring

• expr command can be used to extract a string enclosed by the escape characters "\(" and "\)".

Syntax:

$expr "string" : "\(substring \)"

Example:

$ expr “vemanait” : "ve \(man \)ait" // outputs 'man'

OR

$ expr “vemanait” : "**..** \(**…** \)**…**" // outputs 'man'

VTUSYNC.IN

iii) Locating Position of a Character

• expr command can be used to find the location of the first occurrence of a character inside a string.

Syntax:

expr "string" : "[^ch]*ch" //ch → character

Example:

$ expr “vemana” : "[^a]*a" // outputs 4

while Statement

• while loop can be used to execute a set of statements repeatedly as long as a given condition is true.

Syntax:

while condition is true

do

execute statements

done

• The statements enclosed between do and done are executed repeatedly as long as condition is true.

Example: A script to display a message 3 times using while loop.

```
#! /bin/sh

num=1

while [ $num -le 3 ]

do

        echo " Welcome to Shell Programming "

         num =`expr $num +1`

done
```

VTUSYNC.IN

Output:

Welcome to Shell Programming

Welcome to Shell Programming

Welcome to Shell Programming

Program : To check for a pattern in the file and repeat the process using case and while

```
#!/bin/sh
#searching for a pattern using while loop
answer=y
while [ $answer = y ]
do
        echo "enter the pattern "
        read pname
        echo "enter the file "
        read fname
        grep $pname $fname
        echo "do u wish to continue y or n"
        read anymore
        case $anymore in
                y*|Y*)answer=y ;;
                n*|N*)answer=n;;
                *)answer=exit;;
        esac
done
```

VTUSYNC.IN

for Statement

• for loop can be used to iterate over all items(or strings) within a list.

Syntax:

for variable in list

do

statements

done

• Here, list consists of a set of items(or strings).

• Each item of the list is picked up and assigned to the "variable".

• The iteration continues until all items are picked from the array.

Example: A script to display elements of an array.

```
#! /bin/sh
print("Here are the numbers in the list: \n");
for i in 10 20 30 40 50 60;
do
        echo “$i”
done
```

Output:

Here are the numbers in the list:

10 20 30 40 50 60

Possible Sources of List

1) List from variables

2) List from command substitution

VTUSYNC.IN

3) List from wildcards and

4) List from positional parameters

1) List from Variables

Example: A script to evaluate & display a set of variables using for-loop.

```
#! /bin/sh
x="Dream" y="Believe " z="Achieve"
for var in $x $y $z;
do
        echo "$var \t"
done
```

Output:

Dream Believe Achieve

2) List from Command Substitution

• Command substitution can be used for creating a list.

• Useful: when list is large.

Example: A script to display current date using for-loop.

```
#! /bin/sh
for var in `date` do
        echo "$var \t"
done
```

Output:

Mon Nov 4 08:02:45 IST 2017

3) List from Wildcards

VTUSYNC.IN

• The shell can use wildcards for matching filenames.

Example: A script to print all files with pdf extension.

```
#! /bin/sh
for file in *.pdf
do
        echo "Printing $file \n"
        lp $file
done
```

Output:

Printing chap1.pdf

Printing chap2.pdf

Printing chap3.pdf

4) List from Positional Parameters

Example: A script (program4.sh) to read & display a positional parameters using for-loop.

```
#! /bin/sh
for var in "$*"                 # even "$@" can be used
do
        echo "$var \t"
Do
```

Run:

$ program4.sh A B C

Output:

A B C

set and shift Commands and Handling Positional Parameters

VTUSYNC.IN

set:

• set command can be used to assign positional parameters ($1, $2 and $3) to command line arguments.

• This command can be used for picking up individual fields from the output of a program.

Example:

$ set 98 23 62

• Here, above line assigns

→ 98 to $1

→ 23 to $2

→ 62 to $3.

• This command can also be used to assign the other parameters $# and $*.

Example:

$ set `date`

$ echo $*

Mon Nov 4 08:02:45 IST 2017

Example:

$ echo “The date today is $2 $3, $6”

The date today is Nov 4, 2017

shift :

• shift command is a shell built-in that operates on the positional parameters.

• Each time shift command is called, it shifts/transfers all the positional parameters down by one.

For example:

$2 becomes $1

$3 becomes $2

VTUSYNC.IN

$4 becomes $3, and so on.

Example:

$ echo "$@" # $@ and $* are interchangeable Mon Nov 4 08:02:45
IST 2017

$ echo $1 $2 $3 Mon Nov 4

$ shift # shifts 1 place

$ echo $1 $2 $3 Nov 4 08:02:45

$shift 2 # shift 2 places

$echo $1 $2 $3 08:02:45 IST 2017

Set -- : Helps Command Substitution

• Problem with set command:

When set command is used with command substitution, the output of the command may begin with a -(hypen). In this case, set command interprets -(hypen) as an option and does not work correctly.

Example:

$set 'ls -l student,lst'

-rwxr-xr--: bad option

Solution: Use --(double hypen) immediately after set command.

$set -- 'ls -l student.lst'

-rwxr-xr-- 2 kumar group 163 Jul 13 21:36 student.lst

here (<<) document

VTUSYNC.IN

• The << symbol can be used to read data from the same file containing the script. This file is called as a here document.

• The term 'here' signifies that the data is here rather than in the file.

• Any command using standard input can also take input from a here document.

Syntax:

command << delimiter

document

delimiter

Example:

$ mailx kumar << MARK

Explore

Dream Discover

MARK

• The string (MARK) is delimiter.

• The shell treats every line delimited by MARK as input to the command mailx.

• kumar at the other end will see 3 lines of message text with the date inserted by command.

• The word MARK itself doesn't show up.

Using Here Document with Interactive Programs:

• A shell script can be made to work non-interactively by supplying inputs through here document.

Example:

$ wc -l << END

Decide Commit Succeed

END

VTUSYNC.IN

3 //outputs number of lines = 3

trap

• trap is a signal handler.

• Whenever the interrupt key (Ctrl+C) is pressed, a signal SIGINT is sent to terminate the shell script.

• However, it is not a good practice. For instance, the user may end up leaving a lot of temporary files on the disk.

• trap command can be used to perform clean up operation when a script receives a terminate signal.

• This command is normally placed at the beginning of the shell script.

Syntax:

trap command_list signal_list

• The signal_list contains the signal names (SIGINT, SIGTERM, SIGQUIT).

• The command_list contains the commands to be executed when the signals are received by the script.

• Two common uses of trap:

1) Clean up temporary files and

2) Ignore signals

1) Cleaning up Temporary Files

• The user can remove some files and then exit if someone tries to abort the script from the terminal.

Example:

$ trap ‘rm temp.txt ; exit’ SIGINT

• Here, a file temp.txt will be automatically removed if a signal SIGINT is received by the script.

2) Ignoring Signals

• A script can be made to ignore a specific signal by using a null command list.

VTUSYNC.IN

Example:

trap ‘ ’ SIGINT

• Here, the script ignores a signal SIGINT when it is received.

VTUSYNC.IN